4.3 Gated recurrent units (GRU)

- we have introduced the basics of RNNs, which can better handle sequence data
- however, such techniques may not be sufficient for a wide range of sequence learning problems from nowadays
- for instance, a notable issue in practice is the numerical instability of RNNs
- although we have discussed implementation tricks such as gradient clipping, this issue can be alleviated further with more sophisticated designs of sequence models
- specifically, gated RNNs are much more common in practice

4.3 Gated recurrent units (GRU)

- we will introduce two such widely-used networks, namely gated recurrent units (GRUs) and long short-term memory (LSTM)
- furthermore, we will expand the RNN architecture with a single hidden layer, which has been discussed so far, and describe deep architectures, with multiple hidden layers
- such expansions are frequently adopted in modern recurrent networks

4.3 Gated recurrent units (GRU)

- in Chapter 2, we discussed that long products of matrices can lead to vanishing or exploding gradients
- let us briefly think about what such gradient anomalies mean in practice:
  - We might encounter a situation where an early observation is highly significant for predicting all future observations. Consider the case where the first observation contains a checksum and the goal is to discern whether the checksum is correct at the end of the sequence. In this case, the influence of the first token is vital. We would like to have some mechanisms for storing vital early information in a *memory cell*. Without such a mechanism, we will have to assign a very large gradient to this observation, since it affects all the subsequent observations.
  - We might encounter situations where some tokens carry no pertinent observation. For instance, when parsing a web page, there might be auxiliary HTML code that is irrelevant for the purpose of assessing the sentiment conveyed on the page. We would like to have some mechanism for *skipping* such tokens in the latent state representation.
  - We might encounter situations where there is a logical break between parts of a sequence. For instance, there might be a transition between chapters in a book, or a transition between a bear and a bull market for stocks. In this case, it would be nice to have a means of *resetting* our internal state representation.

4.3 Gated recurrent units (GRU)

- a number of methods have been proposed to address this
- one of the earliest is long short-term memory, proposed in 1997, which we will discuss in Section 4.4
- the gated recurrent unit (GRU), proposed in 2014, is a slightly lighter variant that often offers comparable performance and is significantly faster to compute
- due to its simplicity, we will start with the GRU

4.3 Gated recurrent units (GRU)

- the key distinction between vanilla RNNs and GRUs is that the latter support gating of the hidden state
- this means that we have dedicated mechanisms for when a hidden state should be *updated*, and also when it should be *reset*
- these mechanisms are learned, and they address the concerns listed above
- for instance, if the first token is of great importance, we will learn not to update the hidden state after the first observation
- likewise, we will learn to skip irrelevant temporary observations
- lastly, we will learn to reset the latent state whenever needed; we discuss this in detail below

4.3 Gated recurrent units (GRU)

- the first thing we need to introduce are the *reset gate* and the *update gate*
- we engineer them to be vectors with entries in $(0, 1)$, such that we can perform convex combinations
- for instance, a reset gate would allow us to control how much of the previous state we might still want to remember
- likewise, an update gate would allow us to control how much of the new state is just a copy of the old state

4.3 Gated recurrent units (GRU)

- we begin by engineering these gates
- Figure 5 illustrates the inputs for both the reset and update gates in a GRU, given the input of the current time step and the hidden state of the previous time step
- the outputs of the two gates are given by two fully-connected layers with a sigmoid activation function

4.3 Gated recurrent units (GRU)

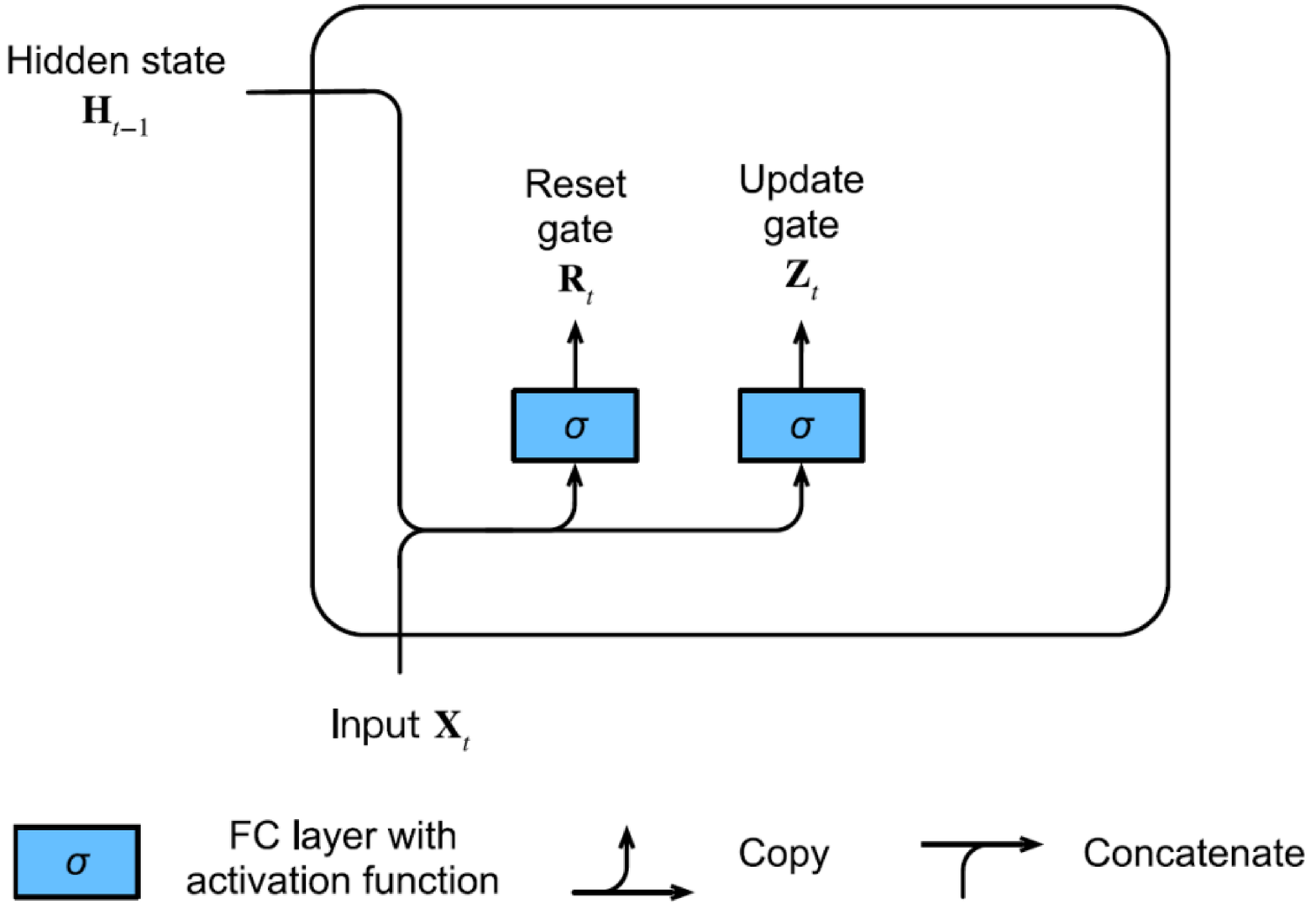


Figure 5: Computing the reset gate and the update gate in a GRU model.

4.3 Gated recurrent units (GRU)

- mathematically, for a given time step t, suppose that the input is a mini-batch $\boldsymbol{X}_t \in \mathbb{R}^{n \times d}$ (number of examples: n, number of inputs: d) and the hidden state of the previous time step is $\boldsymbol{H}_{t-1} \in \mathbb{R}^{n \times h}$ (number of hidden units: h)
- then, the *reset gate* $\boldsymbol{R}_t \in \mathbb{R}^{n \times h}$ and *update gate* $\boldsymbol{Z}_t \in \mathbb{R}^{n \times h}$ are computed as follows:

$$\begin{aligned} \boldsymbol{R}_t &= \sigma(\boldsymbol{X}_t \boldsymbol{W}_{xr} + \boldsymbol{H}_{t-1} \boldsymbol{W}_{hr} + \boldsymbol{b}_r), \\ \boldsymbol{Z}_t &= \sigma(\boldsymbol{X}_t \boldsymbol{W}_{xz} + \boldsymbol{H}_{t-1} \boldsymbol{W}_{hz} + \boldsymbol{b}_z), \end{aligned}$$

  where $\boldsymbol{W}_{xr}, \boldsymbol{W}_{xz} \in \mathbb{R}^{d \times h}$ and $\boldsymbol{W}_{hr}, \boldsymbol{W}_{hz} \in \mathbb{R}^{h \times h}$ are weight parameters, and $\boldsymbol{b}_r, \boldsymbol{b}_z \in \mathbb{R}^{1 \times h}$ are biases
- note that broadcasting is triggered during the summation
- we use the sigmoid function σ to transform input values to the interval $(0, 1)$

4.3 Gated recurrent units (GRU)

- next, let us integrate the reset gate $\boldsymbol{R}_t$ with the regular latent state updating mechanism in (3)
- it leads to the following *candidate hidden state* $\tilde{\boldsymbol{H}}_t \in \mathbb{R}^{n \times h}$ at time step t:

$$\tilde{\boldsymbol{H}}_t = \tanh(\boldsymbol{X}_t \boldsymbol{W}_{xh} + (\boldsymbol{R}_t \odot \boldsymbol{H}_{t-1}) \boldsymbol{W}_{hh} + \boldsymbol{b}_h), \tag{5}$$

  where $\boldsymbol{W}_{xh} \in \mathbb{R}^{d \times h}$ and $\boldsymbol{W}_{hh} \in \mathbb{R}^{h \times h}$ are weight parameters, $\boldsymbol{b}_h \in \mathbb{R}^{1 \times h}$ is the bias, and the symbol $\odot$ is the Hadamard (element-wise) product operator
- here, we use a nonlinearity in the form of tanh to ensure that the values in the candidate hidden state remain in the interval $(-1, 1)$

4.3 Gated recurrent units (GRU)

- the result is a *candidate*, since we still need to incorporate the action of the update gate
- comparing with (3), now, the influence of the previous states can be reduced with the element-wise multiplication of $\boldsymbol{R}_t$ and $\boldsymbol{H}_{t-1}$ in (5)
- whenever the entries in the reset gate $\boldsymbol{R}_t$ are close to 1, we recover a vanilla RNN, such as in (3)
- for all entries of the reset gate $\boldsymbol{R}_t$ that are close to 0, the candidate hidden state is the result of an MLP with $\boldsymbol{X}_t$ as the input
- any pre-existing hidden state is thus *reset* to defaults
- Figure 6 illustrates the computational flow after applying the reset gate

4.3 Gated recurrent units (GRU)

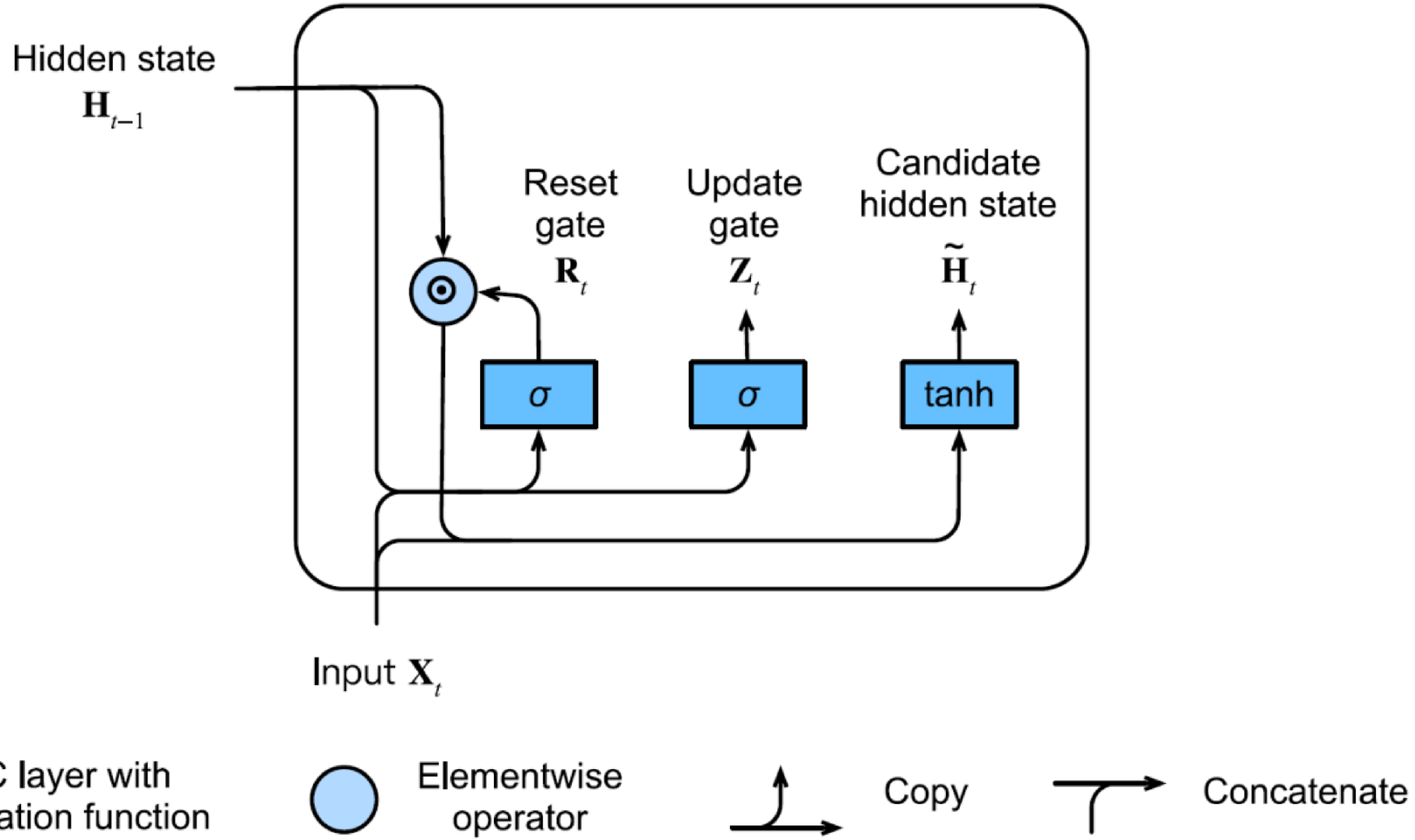


Figure 6: Computing the candidate hidden state in a GRU model.

4.3 Gated recurrent units (GRU)

- finally, we need to incorporate the effect of the update gate $\boldsymbol{Z}_t$
- this determines the extent to which the new hidden state $\boldsymbol{H}_t \in \mathbb{R}^{n \times h}$ is just the old state $\boldsymbol{H}_{t-1}$, and by how much the new candidate state $\tilde{\boldsymbol{H}}_t$ is used
- the update gate $\boldsymbol{Z}_t$ can be used for this purpose, simply by taking element-wise convex combinations between both $\boldsymbol{H}_{t-1}$ and $\tilde{\boldsymbol{H}}_t$
- this leads to the final update equation for the GRU:

$$\boldsymbol{H}_t = \boldsymbol{Z}_t \odot \boldsymbol{H}_{t-1} + (1 - \boldsymbol{Z}_t) \odot \tilde{\boldsymbol{H}}_t.$$

- whenever the update gate $\boldsymbol{Z}_t$ is close to 1, we simply retain the old state
- in this case, the information from $\boldsymbol{X}_t$ is essentially ignored, effectively skipping time step t in the dependency chain

4.3 Gated recurrent units (GRU)

- in contrast, whenever $\boldsymbol{Z}_t$ is close to 0, the new latent state $\boldsymbol{H}_t$ approaches the candidate latent state $\tilde{\boldsymbol{H}}_t$
- these designs can help us cope with the vanishing gradient problem in RNNs and better capture dependencies for sequences with large time step distances
- for instance, if the update gate has been close to 1 for all the time steps of an entire subsequence, the old hidden state at the time step of its beginning will be easily retained and passed to its end, regardless of the length of the subsequence
- Figure 7 illustrates the computational flow after the update gate is in action
- in summary, GRUs have the following two distinguishing features:
  - Reset gates help capture short-term dependencies in sequences.
  - Update gates help capture long-term dependencies in sequences.

4.3 Gated recurrent units (GRU)

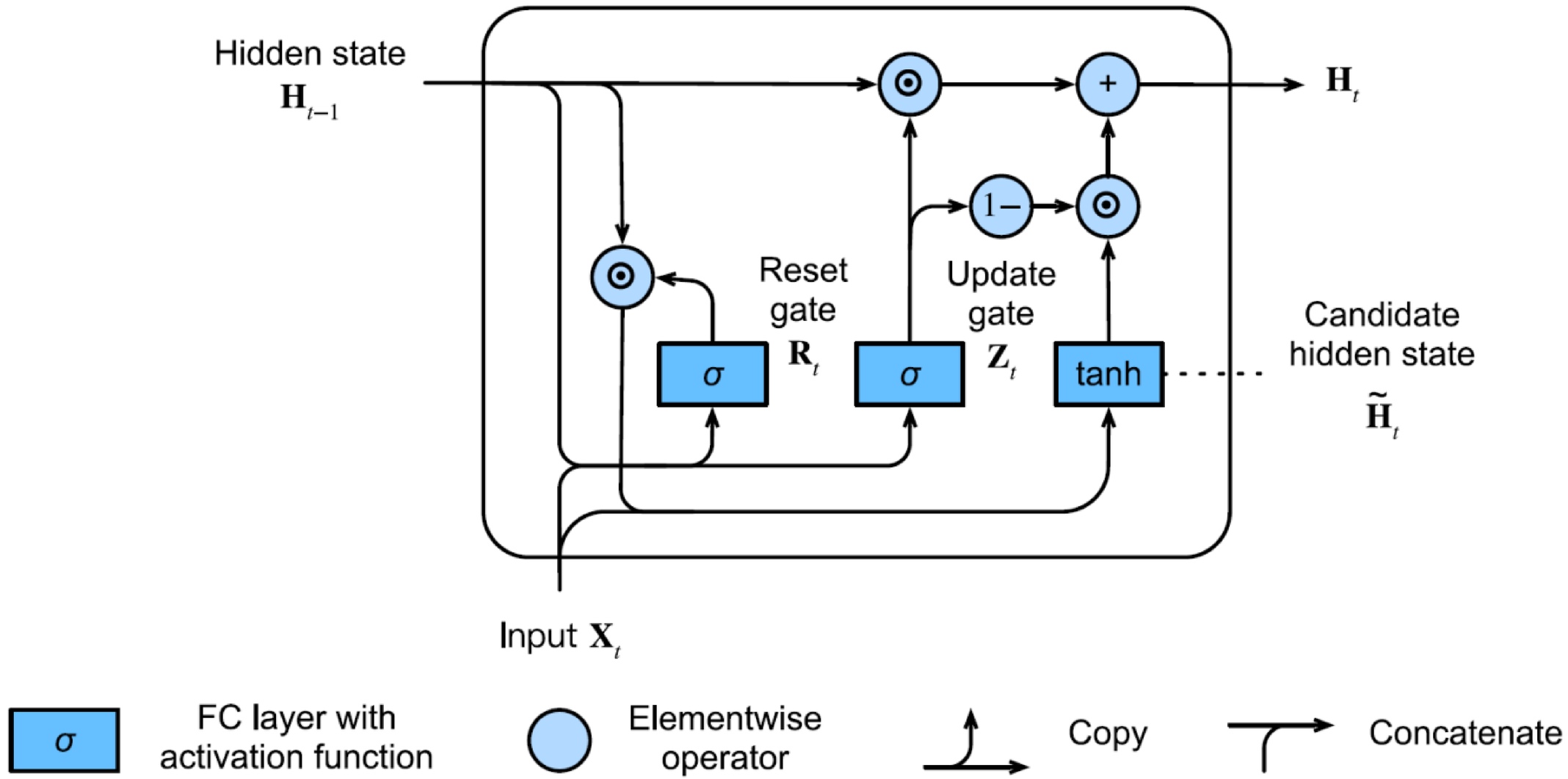


Figure 7: Computing the hidden state in a GRU model.